FaultPilot: Debugging Python AI Applications with Evidence

Phase 1: Literature Review, Research Gaps and Project Proposal

Fazal Zaman Khan (22I-2362)

9 October 2026

Course: AI4015 Agentic Artificial Intelligence
Project code: G2-08
Project seed: Python AI/ML fault diagnosis and repair
Group: Solo, Fazal Zaman Khan (22I-2362)
Supervisor: Meemon Khan (G2 projects, as listed in the course brief)
Repository: https://github.com/FazalZamanKhan/faultpilot-g2-08

Contents

A. Literature Review and Research Gaps **2**
A.1 Scope and selection . . . 2
A.2 Critical paper reviews . . . 2
1. RepairAgent, the base paper [1] . . . 2
2. UniDebugger [2] . . . 2
3. COAST and DEBUGEVAL [3] . . . 2
4. SynFix [4] . . . 3
5. MLDebugging [5] . . . 3
6. Hierarchical Knowledge Injection [6] . . . 3
7. Agent trajectory study [7] . . . 4
8. GraphGlue [8] . . . 4
9. RepoDebug [9] . . . 4
10. DSDBench [10] . . . 5
A.3 Cross-paper comparison . . . 5
A.4 Three selected research gaps . . . 7

B. Project Proposal **8**
B.1 Project, problem and target user . . . 8
B.2 Base paper, extension and agentic justification . . . 8
B.3 Confirmed data and Phase 1 baseline . . . 8
B.4 System architecture and data flow . . . 9
B.5 Research questions, hypotheses and success criteria . . . 11
B.6 Evaluation protocol . . . 12
B.7 Milestones, resources and feasibility . . . 12
B.8 Potential publication direction . . . 13

References **14**

A. Literature Review and Research Gaps

A.1 Scope and selection

This review looks at systems that diagnose or repair code and asks what they can teach us about Python AI applications. The main concerns are how an agent chooses its next action, finds the cause of a failure, uses project context, and checks whether a change worked. The ten papers appeared in peer reviewed ICSE, ASE, EMNLP, or ACL proceedings in 2025. Their results come from different tasks and test suites, so a Java repair count cannot be compared directly with a Python diagnosis score. *RepairAgent* [1] is the base paper because its loop of reading, editing, and testing code offers a concrete starting point. It appeared in the ICSE 2025 Research Track; ICSE has an A* rank in CORE 2023 [11], [12].

A.2 Critical paper reviews

1. RepairAgent, the base paper [1]

RepairAgent asks a useful question: can a language model decide what to inspect and test while it works on a bug? Bouzenia, Devanbu, and Pradel give it tools to read code, search for useful examples, write a patch, and run tests. A state machine controls when those tools are available. Across 835 Java bugs in Defects4J, the system produced 186 patches that passed the supplied tests; the authors judged 164 of them correct. In a 100 bug ablation, correct fixes fell from 21 to 11 when search tools were removed and to 14 when the state machine was removed. A separate GitBug-Java experiment found 13 correct fixes in 100 cases. The average cost was roughly 270,000 model tokens per bug, which makes the method expensive to reproduce at scale.

The paper notes several threats: training data might contain benchmark solutions, the system needs existing failing tests, fault localization can be wrong, and model runs vary. There is also a transfer problem for this project. Python AI scripts can fail because of package versions, tensor dimensions, data values, or execution order, none of which is well represented by a curated Java test suite. FaultPilot will borrow the controlled tool loop but measure it on the same Python cases as a simpler rule based baseline. RepairAgent's Java score is a design reference, not a target to claim as beaten.

2. UniDebugger [2]

UniDebugger divides debugging into connected roles for locating a fault, explaining it, and preparing a patch. The agents work in a hierarchy and can call tools rather than relying only on their initial prompt. Lee and colleagues evaluate the system on Defects4J, Codeflaws, QuixBugs, and ConDefects. For Defects4J they report 286 patches that pass tests, of which 197 are correct, a 25.48% increase in correct fixes over their strongest comparator. On Codeflaws, 3,130 of 3,982 programs receive plausible fixes. A manual check found 95 correct patches in a sample of 100. That sample is useful, but it does not turn all 3,130 plausible fixes into confirmed correct ones.

The authors limit their study to bugs with tests. A passing suite can still miss a broken behavior, and the main repository result is from Java rather than Python AI code. The hierarchy also adds work that may be unnecessary when a traceback already identifies an obvious dependency error. FaultPilot can take the useful separation between diagnosis and repair without sending every case through the full agent chain. A later comparison will check whether extra agent steps actually help on Python cases when a simpler baseline has the same evidence and budget.

3. COAST and DEBUGEVAL [3]

COAST is relevant because a good repair score can hide a weak diagnosis. Yang and colleagues introduce DEBUGEVAL, a set of 5,712 examples in Python, C++, and Java. It tests fault location, bug identification,

repair, and recognition separately. COAST uses quizzer, learner, and teacher agents to create training data for smaller models. In Table 4, the DeepSeek-Coder 6.7B model's average across tasks rises from 28.7 without further training to 56.4 after COAST training. Its repair subscore moves from 40.6 to 49.5. The difference between these figures shows why a single overall number is not enough to describe debugging ability.

The paper also describes limits that matter here. DEBUGEVAL covers four tasks, and the generated examples depend on the quality of the quizzer and teacher. Many examples resemble programming contest snippets, where the package environment is much simpler than a real Python AI project. FaultPilot does not need to copy COAST's training method for Phase 1. Its more useful lesson is how to evaluate the stages independently: first ask whether the fault was identified, then whether the evidence was relevant, and finally whether a proposed change passed meaningful checks.

4. SynFix [4]

SynFix looks at bugs whose fix may involve more than one related file or symbol. It builds a RelationGraph covering classes, functions, variables, and dependencies, then uses that graph to find the likely fault and guide edits. Regression tests provide feedback on the patch. Tang and colleagues report resolving 157 of 300 SWE-bench Lite issues, or 52.33%, at an average model cost of USD 0.56 per issue. They also report 55.8% on SWE-bench Verified and 29.86% on the full benchmark. These are issue resolution results under the benchmark's tests; they do not measure how well the system explains a root cause.

The method depends on two things being reliable: the graph of code relationships and the regression tests. The authors acknowledge that domain specific code and poor documentation can weaken those inputs. The study does not settle what happens when a Python library version changes or when an application has no trustworthy test suite. For a solo project, SynFix suggests a manageable alternative to long tool loops: retrieve only the relationships that matter, then check the proposed change. FaultPilot will compare that approach with a single model call and retain links to the code and test evidence behind each diagnosis.

5. MLDebugging [5]

MLDebugging is the closest available benchmark to the setting of this project. Huang and colleagues release 1,175 Python cases involving 126 libraries and seven fault categories. Each record includes buggy code, reference code, tests, runtime feedback, and a label. In Table 5, DeepSeek-V3 reaches an average test pass rate of 58.7%, while the 7B Qwen2.5 model reaches 42.7%. Import errors and misunderstood requirements remain difficult. These results suggest that even a strong model cannot reliably solve code that combines several libraries from a traceback alone.

The authors note that many examples were generated with models and that recreating the software environments takes work. I also checked the pinned public data file. There are 31 records contain more than one recognizable category mention, and two have no runtime feedback. Those counts come from the repository profile, not from the published paper. They matter because forcing every record into exactly one class could distort evaluation. FaultPilot will use this dataset as its primary source, keep the ambiguous labels visible, and split related examples by their source task ID. The current baseline has only been run on four synthetic cases, so it has no MLDebugging score yet.

6. Hierarchical Knowledge Injection [6]

Ehsani and colleagues test how much background information a model needs to repair a Python bug. Their method adds context in stages: facts near the bug, relationships and history from the repository, then project documentation and earlier fixes. With Llama 3.3 on 314 BugsInPy cases, the number of fixes rises from 207 with local bug information to 235 with repository information and 250 with project information.

The final 250 out of 314 is about 79% by their test criterion. Table II has a revealing counterexample: giving all knowledge layers at once produces only 207 fixes. Extra context can therefore be a distraction rather than a benefit.

The labels for bug types were assigned manually, and passing the supplied tests does not prove that a fix is correct in every setting. The paper also leaves difficult program anomaly and GUI cases unresolved. Its numbers should not be placed beside RepairAgent's as though the models and test conditions were identical. The useful idea for FaultPilot is to ask for more context only when the traceback and local code leave a question open. An ablation can show whether that extra retrieval improves diagnosis enough to justify its time and token cost.

7. Agent trajectory study [7]

Bouzenia and Pradel examine the actions that repair agents take, rather than only counting the final patches. They turn logs from RepairAgent, AutoCodeRover, and OpenHands into 120 trajectories with 2,822 model interactions. The average number of iterations varies a great deal: 34 for RepairAgent, 29 for OpenHands, and 6 for AutoCodeRover. Average token use is roughly 220,000, 1.2 million, and 23,000 respectively. The authors find repeated actions and misread feedback in unsuccessful attempts. Those observations help explain where time can go, although this study is not itself a new repair benchmark.

Only three agents were studied, and deciding whether an action was useful involves human judgment. The agents also use different models and tasks. Their raw iteration counts should therefore not be treated as a controlled comparison of efficiency. For FaultPilot, the practical takeaway is to record every tool call and the evidence it produced. A retry should have a reason, such as a new failing test; repeating the same inspection should count against the budget. The later evaluation can measure repeated calls and wasted attempts, and see whether a clear stopping rule reduces them.

8. GraphGlue [8]

GraphGlue deals with a failure that is specific to deep learning code. Some Python constructs prevent a compiler from capturing a complete computation graph, even when the program otherwise runs. Wang and colleagues use several agents to find graph break causes, suggest code changes, and reject changes that do not help. The ASE 2025 research paper record reports that GraphGlue optimized 92.63% of 1,411 real user programs. Compared with TorchDynamo alone, the reported speedup reached 2.19 times at best and averaged 1.23 times. The abstract also reports an average memory saving of 8.74 times against the selected compiler frontend. These are optimization results, not a general measure of bug repair.

The link to FaultPilot is the use of an executable check after a diagnosis. A convincing explanation is not enough if the program still behaves the same way. At the same time, graph capture is a narrow problem. These results do not show that GraphGlue can diagnose missing packages, device mismatches, or faulty data transformations. FaultPilot will leave graph breaks as a possible extension and use the broader lesson in its main workflow: a proposed cause should lead to a check that could prove it wrong.

9. RepoDebug [9]

RepoDebug asks how models cope when a bug sits inside a repository rather than a short code snippet. Liu and colleagues build cases from 63 public repositories, inject 22 types of errors across eight languages, and test identification, localization, and repair. They evaluate ten models. In the Python results in Table 3, Claude 3.5 Sonnet identifies the bug type in 36.77% of cases, locates the single bug in 8.17%, and passes the repair test in 6.15%. These are separate outcomes. Calling the type correctly is far easier than producing a working patch. The paper also reports a decline as the supplied code context grows.

The errors were injected, so their pattern may not match the failures people encounter in AI applications. The authors discuss the difficulty of using long repository context and the computing cost of these tests. Only part of the corpus received a manual quality check, which also limits confidence in the aggregate figures. FaultPilot will start with Python cases, but it should still measure diagnosis separately from repair and retain the surrounding code that explains a failure. RepoDebug also argues against sending an entire repository to a model when a smaller, relevant evidence window may be enough.

10. DSDBench [10]

DSDBench focuses on data science scripts in which the line that crashes is not necessarily the line that caused the problem. Yang and colleagues take tasks from data science benchmarks, insert errors, record runtime messages, and label both cause and effect lines. The 1,117 cases include 741 with one error and 376 with several; cases in the second group average 2.87 errors. Table 3 shows the difficulty of combined faults. GPT-4o finds the cause line in 39.0% of cases with one error but only 20.3% of cases with several errors. Its accuracy on the effect line falls from 34.3% to 10.4%. The benchmark also checks error type and message quality, rather than reducing debugging to patch success.

The authors note that the benchmark is limited to data science scripts, not larger projects with several files. Its inserted faults and partly model assisted message rubric may also behave differently from real incidents. For FaultPilot, the important distinction is between a visible symptom and the earlier mistake that produced it. DSDBench could serve as a second test set for cause localization. It would also test whether a verification step catches a partial fix when another bug remains in the program.

A.3 Cross-paper comparison

The studies report different outcomes: patches judged correct, patches that pass the available tests, and diagnostic scores. Their test suites and denominators differ, so the figures do not form one ranking.

Table 1: Comparison of the ten papers. The star marks the base paper.

#	Title	Year / venue	Dataset	Method / agent architecture	Evaluation metrics	Key result	Limitation / research gap
1⋆	RepairAgent [1]	2025 ICSE	Defects4J (835); GitBug-Java (100)	Model tools and state machine	Correct fixes; token cost	164/835 correct; 13/100 correct	Java setting; existing tests; cost
2	UniDebugger [2]	2025 EMNLP	Defects4J; Codeflaws; QuixBugs	Agents for diagnosis and repair	Correct and plausible fixes	197 correct; 286 plausible on Defects4J	Needs tests; added agent cost
3	COAST [3]	2025 NAACL Findings	DEBUGEVAL (5,712)	Agents create training data	Scores for each debugging task	6.7B model average: 28.7 to 56.4	Depends on synthetic data quality
4	SynFix [4]	2025 ACL Findings	SWE-bench Lite and others	Dependency graph; regression tests	Resolved issues; cost; tokens	157/300 Lite; USD 0.56 mean	Weak tests or graph hurt validation
5	MLDebugging [5]	2025 ACL Findings	1,175 Python cases; 126 libraries	Benchmark with seven fault classes	Test pass rate	Best average: 58.7%	Generated cases; label and setup noise
6	Knowledge injection [6]	2025 ASE	BugsInPy (314)	Staged local, repo, project context	Fixed bugs; Pass@k	250/314 fixed (79%)	Test pass is a proxy for correctness
7	Agent trajectories [7]	2025 ASE	120 runs; 2,822 actions	Analysis of agent action logs	Steps; tokens; repetition	Mean 6 to 34 steps by agent	Three different agents; correlation only
8	GraphGlue [8]	2025 ASE	1,411 DNN programs	Finds graph breaks; checks edits	Optimization; speed; memory	92.63% optimized; 1.23× mean speed	Limited to compiler graph faults
9	RepoDebug [9]	2025 EMNLP Findings	63 repos; 8 languages	Repository debugging benchmark	ID and location accuracy; repair Pass@1	Python: 36.77%, 8.17%, 6.15%	Injected errors; long context
10	DSDBench [10]	2025 EMNLP	1,117 data science cases	Labels cause and effect lines	Localization and type accuracy	GPT-4o cause: 39.0% single, 20.3% multiple	Inserted faults in scripts

A.4 Three selected research gaps

Gap 1: Evidence for a diagnosis in Python code that uses several libraries. MLDebugging's strongest tested model averages a 58.7% pass rate [5]. RepoDebug finds that identifying a bug is easier than locating or repairing it in Python [9], while DSDBench shows that the line that fails may only be a symptom [10]. FaultPilot will report a fault category, a possible cause, and the code or runtime evidence behind that judgment. It will test whether retrieving library and project context improves macro F1 and cause localization over rules and a single model call. A correct sounding label is of little use if it points to the wrong line.

Gap 2: Checking a repair when tests are incomplete. RepairAgent reports 186 patches that pass tests, but only 164 judged correct [1]. UniDebugger reports 286 plausible fixes and 197 correct ones [2]. SynFix also depends on regression tests, whose coverage may be weak [4]. FaultPilot will first reproduce the failure, make a candidate edit in a disposable workspace, and then run the target and regression checks. Its report will say exactly which checks passed. The phrase *verified by supplied tests* will describe that limited evidence; it will not mean universally correct. We will measure successful target tests, regressions, and how many apparently plausible patches fail stronger checks.

Gap 3: Knowing when to stop or ask for help. Removing RepairAgent's state machine reduced correct fixes from 21 to 14 in its 100 bug ablation [1]. A later trajectory study found that failed RepairAgent runs averaged 40 iterations, compared with 22 for successful runs [7]. In a separate Python study, adding context in stages worked better than giving it all at once: 79% versus 65% in one Llama 3.3 setting [6]. FaultPilot will keep a record of its state, limit retries, and stop with a diagnostic report when the evidence is too weak for a repair. Selective accuracy, unsupported suggestions, tool calls, latency, and token use will show whether that stopping rule helps.

Table 2: Evidence, approach, and measurements for the three selected gaps.

Gap	Supporting evidence	Proposed approach	How it will be measured
1. Grounded diagnosis	MLDebugging [5], RepoDebug [9], DSDBench [10]	Normalize labels; link traceback, code and library evidence to each diagnosis	Macro F1, cause line accuracy, and evidence audit
2. Verified repair	RepairAgent [1], UniDebugger [2], SynFix [4]	Isolated patch; target and regression tests; retain failure traces	Repair test pass rate, regression rate, patches rejected by checks
3. Bounded abstention	RepairAgent [1], trajectories [7], knowledge injection [6]	State machine, retry and tool budget, stop or abstain when evidence is weak	Selective accuracy at 70% coverage, unsupported suggestions, calls, latency, tokens

B. Project Proposal

B.1 Project, problem and target user

Selected seed and title. Project G2-08 is developed as *FaultPilot: Debugging Python AI Applications with Evidence*. The project focuses on Python programs for data science and machine learning. Such programs can fail while importing a package, moving tensors between devices, transforming data, or calling a library. The intended user is a student or developer who has a traceback and relevant code but cannot tell whether the visible error is the cause or a later symptom.

The problem matters because an AI application may combine libraries with different assumptions about shapes, types, and devices. One wrong transformation can cause an error much later in a model call. An assistant that offers a patch without showing its evidence or test results leaves the developer to guess whether the fix is safe. MLDebugging reports modest pass rates for code using several libraries [5], and DSDBench shows that multiple faults are especially hard to locate [10]. FaultPilot is planned as a local command line tool that keeps track of its evidence, makes a limited repair attempt, checks it in a separate workspace, and asks for review when it cannot support a confident answer.

The final tool should accept a case file, repository path, traceback, trusted test command, and optional model configuration. Its JSON report and short plain language summary will include a category, suspected cause location, supporting code or runtime evidence, any proposed edit, the checks run, and one of three statuses: *verified by supplied tests*, *unverified*, or *abstained*. It will leave the user's original repository untouched until a person approves a change. A VS Code interface would be useful later, but the command line tool and its evaluation are the main deliverables.

B.2 Base paper, extension and agentic justification

RepairAgent [1] is the base paper because it describes a complete cycle of choosing tools, editing code, and testing a result. Its ablations also show why search and state control matter. The 164 correct fixes among 835 Java Defects4J bugs support those design choices, but that number is not a fair target for a Python benchmark. FaultPilot will adapt the explicit state, limited tools, and test feedback. It will then study Python code that uses several libraries, require a visible evidence trail, and allow the system to abstain when the evidence is weak. The diagnosis stage will consult a fault taxonomy and relevant local context. The verification stage will list the checks actually run. Repeated evidence will trigger a stop rather than another identical attempt.

The implemented keyword baseline is useful for obvious tracebacks. It cannot decide which other file or library to inspect when the cause is unclear, or respond to a new test failure after trying a patch. A single model call has the same problem because the later test result does not yet exist. Retrieval alone can supply context, but it cannot decide whether to run a reproducer or reject an edit. The proposed agent will make a small number of decisions based on what each step returns: inspect more evidence, form a diagnosis, test an edit, retry with new information, or stop. Those decisions, rather than the length of a prompt, are what make the planned workflow agentic.

B.3 Confirmed data and Phase 1 baseline

The confirmed research source is the public MLDebugging JSONL from [5]. The repository pins it to commit `0fd4dec`; `faultpilot/data.py` records the full commit and SHA-256 checksum. The ingestion command downloaded and checked 1,175 records with buggy and reference code, tests, runtime feedback, labels, and source task IDs. A profile found 31 records with more than one recognizable category mention and two without runtime feedback. The raw data stays at its published source rather than being copied into the repository. Examples with the same `task_id` stay together: a SHA-256 rule puts 234 records in the test group and 941 in development. This split is fixed before comparing models.

The Phase 1 baseline is small but runnable. It reads a local JSON case, matches words in the traceback to shape, device, or dependency faults, and runs a trusted Python reproducer with a timeout of five seconds. Its JSON result contains a category, a hint, the reproduction result, and the status `diagnosis_only_no_repair_attempted`. Four synthetic cases cover the three known categories and an unknown fallback. All four reproduce their expected failures and match their labels; four automated tests pass. This checks the pipeline, **not** research accuracy. The optional Ollama adapter passes a local mock server test of its HTTP and response format. No actual model has been run and the baseline does not generate patches.

The README documents the case format and commands. The published dataset has seven categories, which do not map neatly to the baseline's three named categories. The next step is to document that mapping, keep records with ambiguous labels visible, and only then report a baseline score on the held out set. Otherwise, quiet decisions made during data cleaning could make the proposed method look better than it is.

B.4 System architecture and data flow

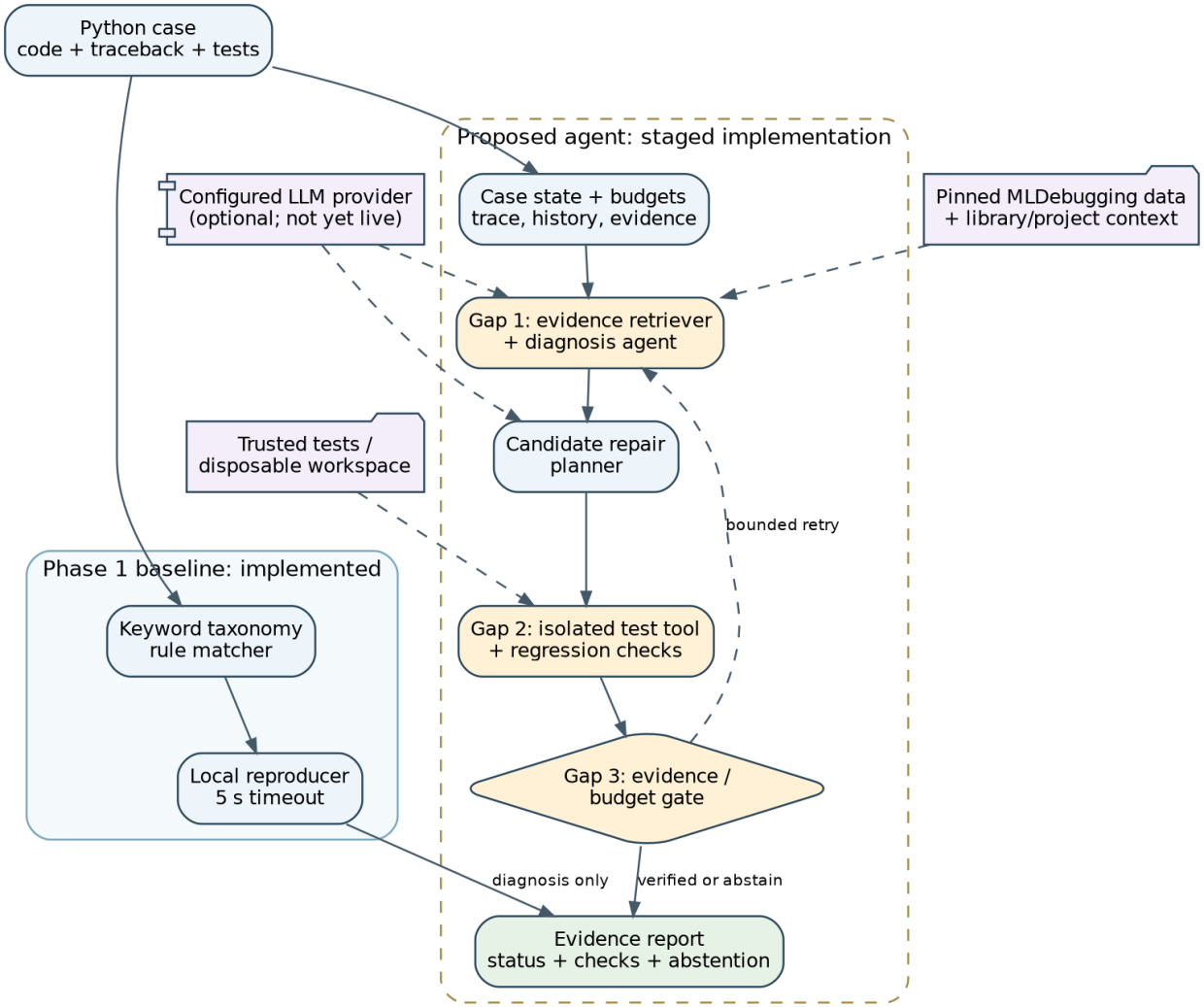


Figure 1. FaultPilot's implemented baseline and proposed agent. Gap 1, Gap 2, and Gap 3 are marked at their intervention points; dashed components are planned.

The planned *case state* will hold the case ID, repository revision, traceback, evidence and its file paths, possible fault categories, diagnosis, candidate patch, test results, budgets, attempt count, and final status. Each action will leave a record that explains the next decision. The available tools will read selected code, retrieve relevant library or project material, write an edit to a disposable copy, and run approved tests. Retrieval will use the repository and versioned local material for that case. Text from a web page will not become an instruction to run a command. A configured model may suggest a diagnosis or edit, but program code will control tool access and stopping rules.

For **Gap 1**, the retriever will collect the relevant traceback frame, nearby code, package information when available, and a few related symbols or passages. A diagnosis must point back to those pieces of evidence. For **Gap 2**, a candidate edit will be tried in a separate workspace. The original failure must reproduce first, and the target and regression tests must pass after the edit; missing tests or failed setup will be reported as unverified. For **Gap 3**, the controller will retry only when a test adds useful new information. Otherwise it will stop and explain what remains uncertain. The left branch of the diagram shows what is already implemented: keyword diagnosis, a reproducer, and a report. The other branches are the planned work.

The user will approve any change before it reaches the original checkout. Test commands will come from an allowlist and have a timeout, and the report will show the evidence used. Model text cannot choose unrestricted shell commands. The present reproducer executes Python `-c` from trusted local fixtures only; it must not be given an external case file or one generated by a model. Later benchmark runs will use disposable environments with pinned dependencies.

B.5 Research questions, hypotheses and success criteria

Gap	Narrow empirical question	Testable prediction and success criterion
1	On the held out MLDebugging cases, does retrieving code and library context improve diagnosis of the seven fault categories over rules and a single model call?	Macro F1 improves by at least 0.08 over the strongest simpler comparator. At least 80% of a 50 case evidence audit links the diagnosis to a relevant source span.
2	With the same number of model attempts, does test feedback improve repair quality over a single candidate?	Target test success improves by at least 10 percentage points on executable cases, while regressions among accepted patches stay at or below 5%. Report confidence intervals even if these targets are missed.
3	Can a limited retry and evidence rule reduce unsupported repair advice at the same answer rate?	At 70% coverage, selective diagnosis accuracy exceeds the always answer variant by at least 0.10. Median tool calls stay at or below eight per case; report token use and latency as well.

These are targets for later experiments, not results already achieved. Some source records lack feedback or mention more than one label. The evaluation will therefore show a complete record count and define the executable subset clearly. Paired comparisons will use the same cases and model budget.

B.6 Evaluation protocol

The 234 held out MLDebugging records are the primary test set. Rules, prompts, and confidence thresholds may be set on the development records; the test group will remain frozen. Category results will include macro F1 plus precision and recall for each class. Records with several labels will follow a declared protocol and be reported separately, rather than disappearing from the denominator. Fifty test cases sampled across categories will be checked by hand against their code, runtime feedback, and source paths. Repair tests will run where a pinned environment can be recreated. A patch that passes the target test but breaks a regression test will count as a regression. Failed dependency setup will be reported with its case count and excluded only from repair rates that require execution.

The comparison has five stages: B0, the rule baseline after its mapping to seven classes; B1, one model call with traceback and local code; B2, that model with conditional retrieval; B3, a limited loop that reads, edits, and tests; and B4, the same loop with an evidence gate and an option to abstain. Model based stages will share a provider, temperature, attempt limit, and token budget. Separate ablations will remove retrieval, test feedback, or abstention. DSDBench could provide a second analysis of cause and effect lines in cases with several errors if its runtime assets work locally. MLDebugging remains the primary source.

The report will include macro F1, cause line accuracy where labels permit it, target test success, regressions, answer coverage, selective accuracy, latency, tool calls, and model tokens and cost. Bootstrap intervals will group cases by their source task so related variants are sampled together. Each ablation will use exactly the same cases. The failure analysis will distinguish wrong categories, missing evidence, environment setup failures, patches that only satisfy a narrow test, and exhausted budgets. The four synthetic Phase 1 cases will remain a smoke check and will not enter the research test set.

B.7 Milestones, resources and feasibility

Stage	Deliverable	Main risk and fallback
Phase 1 foundation	Local case pipeline, rule baseline, tests, optional model adapter checked with a mock server, pinned MLDebugging ingestion and profile, architecture and review	The current result shows that the pipeline runs. It does not establish model or repair quality.
Next 1 to 2 weeks	Normalize benchmark labels; build an executable subset and environment; freeze test groups	Some dependencies may fail. Publish the case counts and use the reproducible subset without changing the held out rule.
Following 2 weeks	Run a configured model; implement single call and retrieval comparisons; attach evidence links	Model access or cost may limit runs. Use a small development sample before one frozen test run with a fixed budget.
Following 2 weeks	Add isolated patch verification, bounded retries, and abstention	Unsafe or flaky tests can give false assurance. Use disposable execution, timeouts, repeated checks, and an `unverified` status.

Stage	Deliverable	Main risk and fallback
Final evaluation and paper	Ablations, confidence intervals, failure analysis, release of scripts and manifests	If full repair remains too costly, report a rigorous diagnosis/abstention study and treat repair as a measured limitation rather than imply success.

This is a solo project. Fazal Zaman Khan is responsible for reading the papers, checking data and licenses, implementation, test environments, evaluation, writing, and submission. Rules and data profiling run on a laptop. Model experiments may need an API or hosted GPU, so calls and tokens will have limits. Secrets will stay out of Git. No live model run was available for Phase 1; testing that connection is a specific next step rather than a result claimed here.

B.8 Potential publication direction

If the controlled experiments support the hypotheses, a possible paper could present a repair method for Python AI faults that records its evidence and works within a tool budget. The reproducible data split, label audit, and analysis of failures could also be useful to others. Results on independent cases beyond MLDebugging would make that claim stronger, especially if the code, prompts, and test setup can be released. A clear negative result would matter too: perhaps retrieval or extra testing costs more than it helps. The four case smoke check cannot support a publication claim.

References

[1] I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An Autonomous, LLM-Based Agent for Program Repair," in *Proc. 2025 IEEE/ACM 47th Int. Conf. Software Engineering (ICSE)*, 2025, pp. 2188–2200, doi: 10.1109/ICSE55347.2025.00157.

[2] C. Lee *et al.*, "UniDebugger: Hierarchical Multi-Agent Framework for Unified Software Debugging," in *Proc. 2025 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 18237–18266, doi: 10.18653/v1/2025.emnlp-main.921.

[3] W. Yang *et al.*, "COAST: Enhancing the Code Debugging Ability of LLMs through Communicative Agent Based Data Synthesis," in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 2570–2585, doi: 10.18653/v1/2025.findings-naacl.139.

[4] X. Tang *et al.*, "SynFix: Dependency-Aware Program Repair via RelationGraph Analysis," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 4878–4894, doi: 10.18653/v1/2025.findings-acl.252.

[5] J. Huang *et al.*, "MLDebugging: Towards Benchmarking Code Debugging Across Multi-Library Scenarios," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 5866–5879, doi: 10.18653/v1/2025.findings-acl.305.

[6] R. Ehsani, E. Parra, S. Haiduc, and P. Chatterjee, "Hierarchical Knowledge Injection for Improving LLM-based Program Repair," in *Proc. 2025 40th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2025, pp. 1440–1452, doi: 10.1109/ASE63991.2025.00122.

[7] I. Bouzenia and M. Pradel, "Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories," in *Proc. 2025 40th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2025, pp. 2846–2857, doi: 10.1109/ASE63991.2025.00234.

[8] H. Wang *et al.*, "Fixing Broken Graphs: LLM-Powered Automatic Code Optimization for DNN Programs," in *Proc. 2025 40th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2025, pp. 1718–1730, doi: 10.1109/ASE63991.2025.00144.

[9] J. Liu *et al.*, "RepoDebug: Repository-Level Multi-Task and Multi-Language Debugging Evaluation of Large Language Models," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 23784–23813, doi: 10.18653/v1/2025.findings-emnlp.1294.

[10] Z. Yang, S. Wang, Y. Yan, and Y. Deng, "Why Stop at One Error? Benchmarking LLMs as Data Science Code Debuggers for Multi-Hop and Multi-Bug Errors," in *Proc. 2025 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 17366–17381, doi: 10.18653/v1/2025.emnlp-main.878.

[11] ICSE 2025, "RepairAgent: An Autonomous, LLM-Based Agent for Program Repair," Research Track program record. [Online]. Available: https://conf.researchr.org/details/icse-2025/icse-2025-research-track/160/RepairAgent-An-Autonomous-LLM-Based-Agent-for-Program-Repair. Accessed: Oct. 9, 2026.

[12] Computing Research and Education Association of Australasia, "CORE 2023 Conference Rankings," entry for International Conference on Software Engineering (ICSE), rank A*. [Online]. Available: https://portal.core.edu.au/conf-ranks/?by=all&page=1&search=&sort=arank&source=CORE2023. Accessed: Oct. 9, 2026.